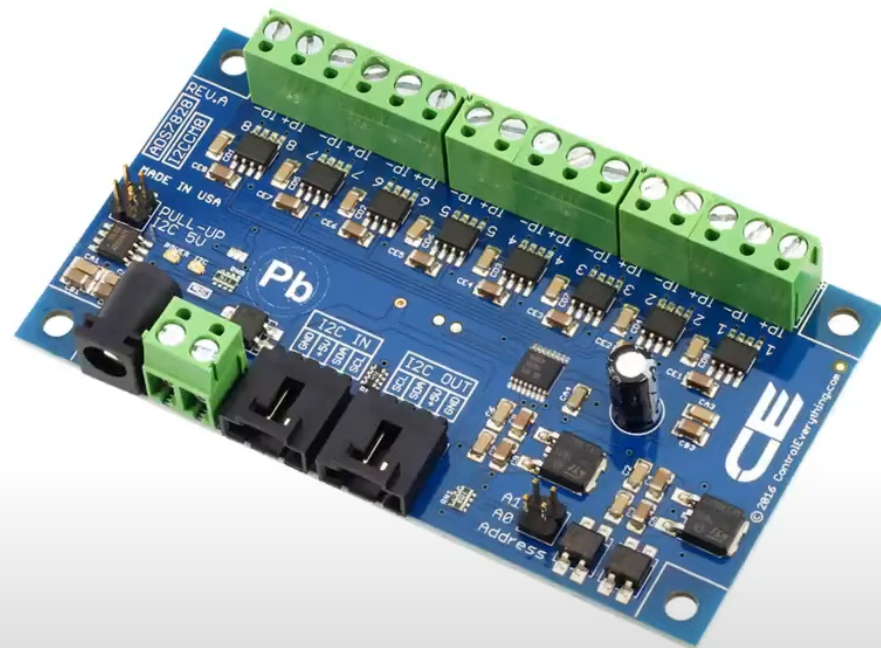


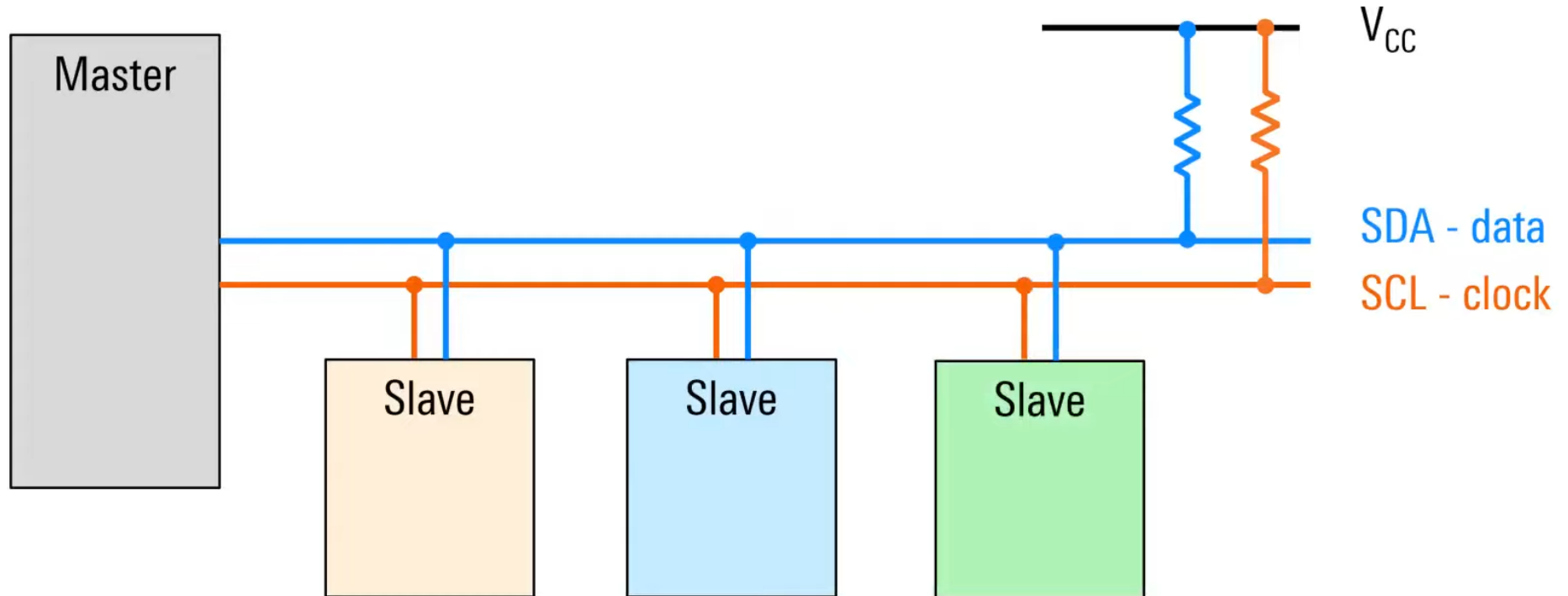
About I2C

- ▶ Inter-Integrated Circuit (I²C or I2C)
 - Developed in 1982 by Philips (now NXP)
 - Very common
 - Primarily used for short distance data communications
- ▶ Synchronous master - slave protocol
 - Both master and slaves can send / receive data
 - Bidirectional, half duplex
 - Can run at different speeds
- ▶ Two wires: serial clock (SCL) and serial data (SDA)



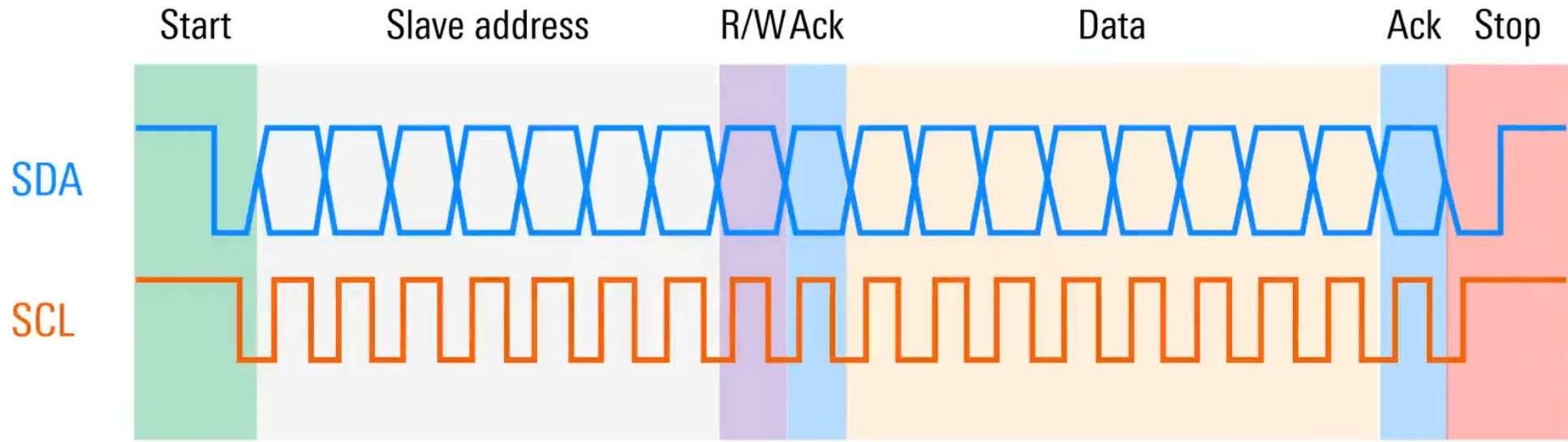
presentation, we'll go step-by-step through the frame structure and operation of I2C.

Basic I2C topology



Note that devices can be added to or removed from the bus at any location and at any time.

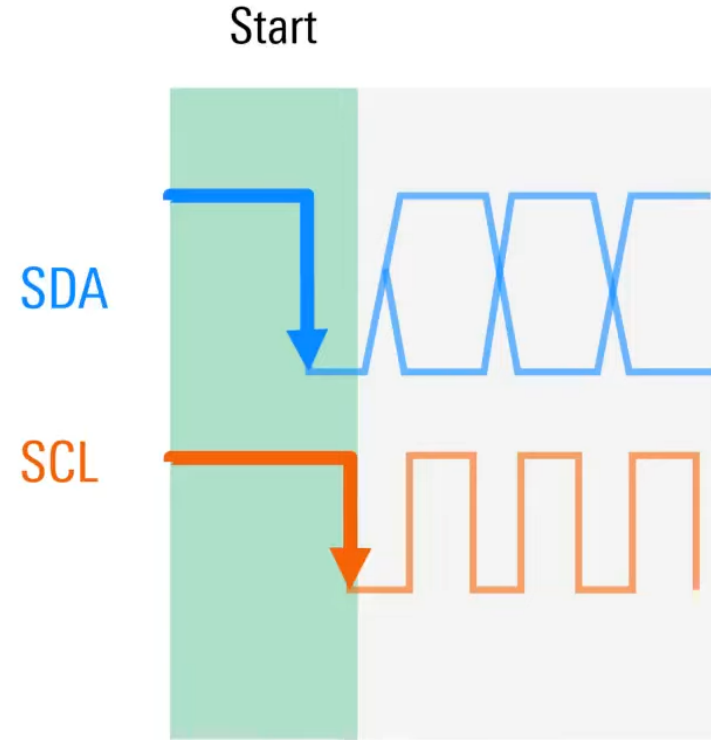
Overview of I2C frames



and acknowledged. The communication is terminated by means of a stop condition.

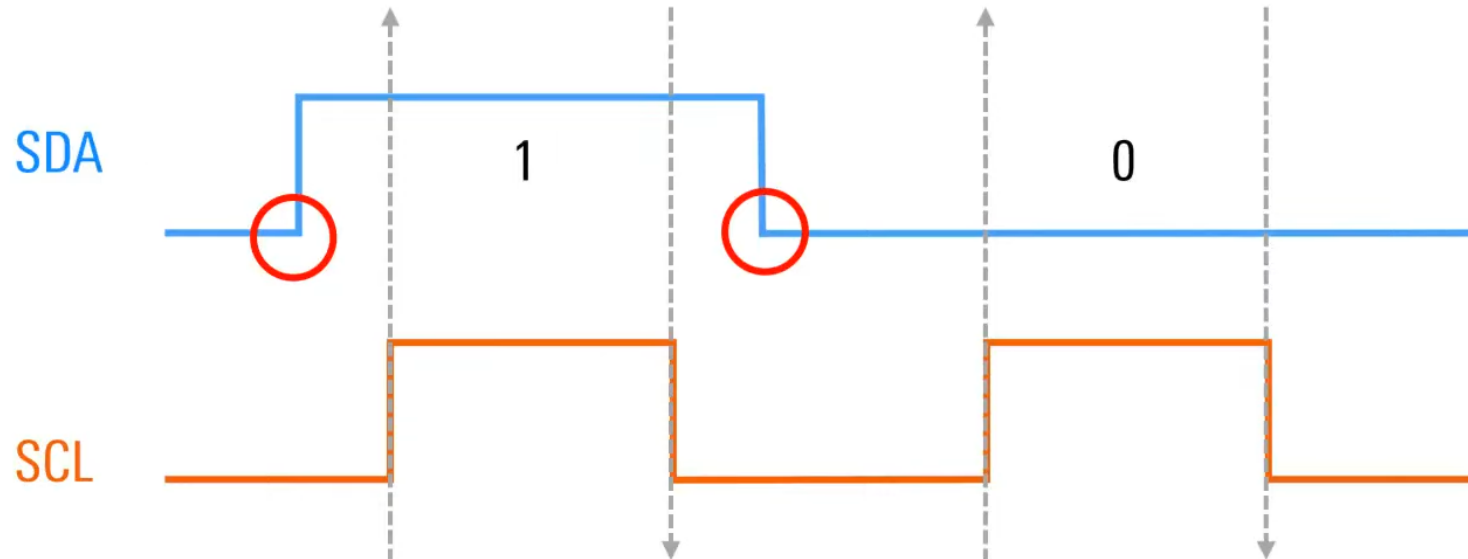
Start condition

- ▶ In the idle state SDA and SCL are both high
- ▶ The **start condition** occurs when a node
 - First pulls SDA low
 - Then pulls SCL low
- ▶ This “claims the bus”
 - Node is now the master
 - Prevents any other nodes from taking control of the bus
 - Reduces the risk of contention
- ▶ Master that has seized the bus also starts the clock



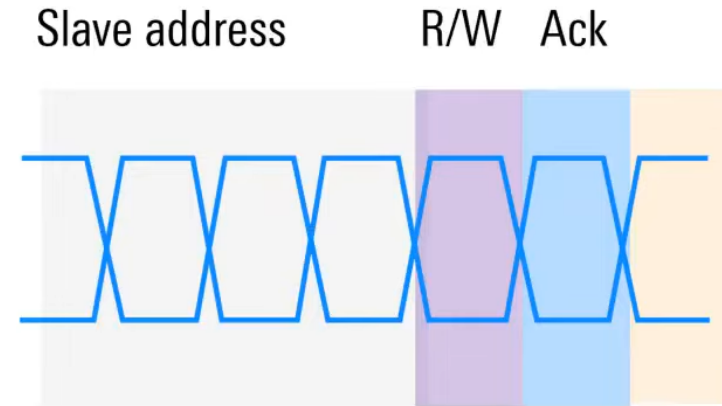
Aside: timing relationship between SDA and SCL

- ▶ SDA does not change between clock rising edge and clock falling edge
- ▶ During **data** transmission, SDA only transitions while SCL is **low**
 - An SDA transition when SCL is **high**, indicates a **start or stop** condition



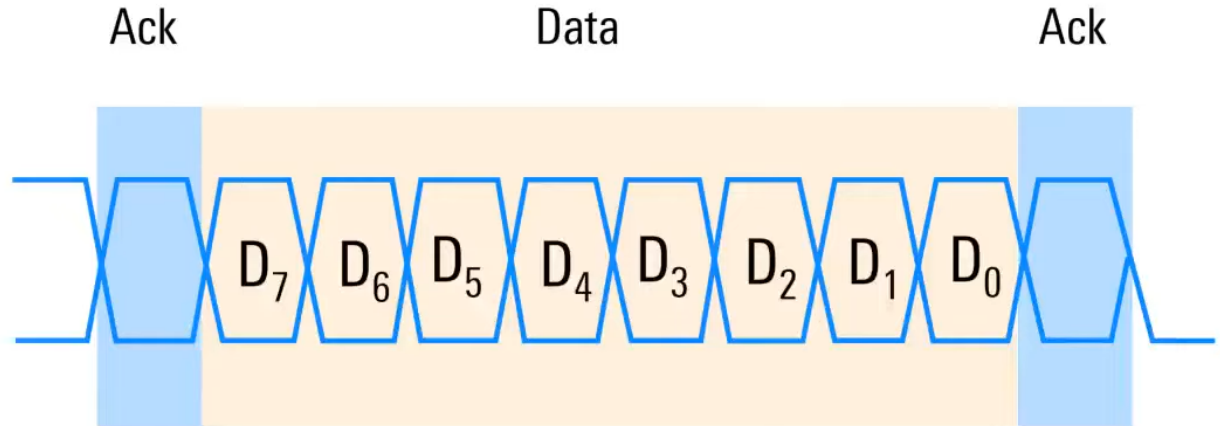
Ack(knowledge) bit

- ▶ Sent by the receiver of a byte of data
 - 0 → acknowledgement (ACK)
 - 1 → negative acknowledgement (NACK)
- ▶ Recall that I2C is idle high
 - Lack of response = NACK
- ▶ Used after slave address and each data byte
- ▶ ACK after data byte(s) confirms receipt of data
- ▶ ACK after slave address confirms that
 - A slave with that address is on the bus
 - The slave is ready to read / write data (depending on R/W bit)



Data byte(s)

- ▶ Data byte contains the information being transferred between master and slave
 - Memory or register contents, addresses, etc.
- ▶ Always 8 bit long. MSB first
- ▶ Always followed by an acknowledgement bit
 - Set to zero by the receiver if data has been received properly

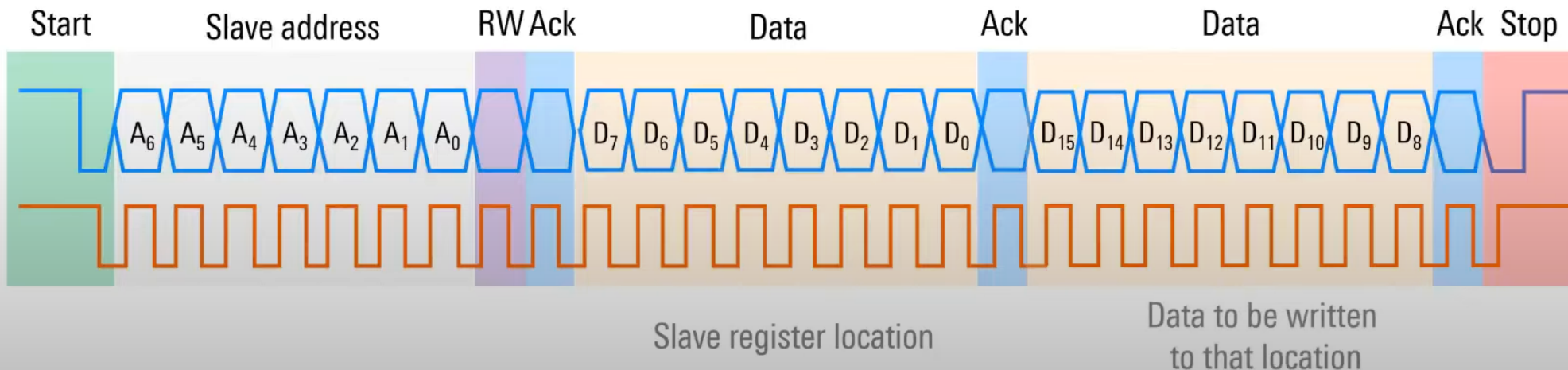


slide - is set to zero by the receiver

Understanding I2C when the data has been properly received.

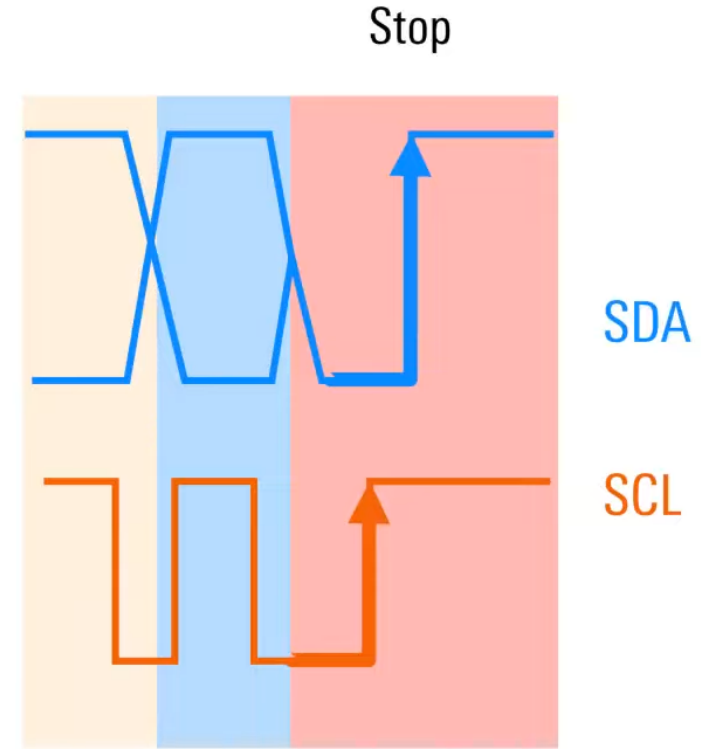
Multiple data bytes

- ▶ In many cases, multiple data bytes are sent in one I2C frame
 - Each data byte is followed by an ACK bit
- ▶ Bytes may be all “data” or some may represent an internal address, etc.
 - Ex: first byte is a register location and second byte is the data to be written to that register



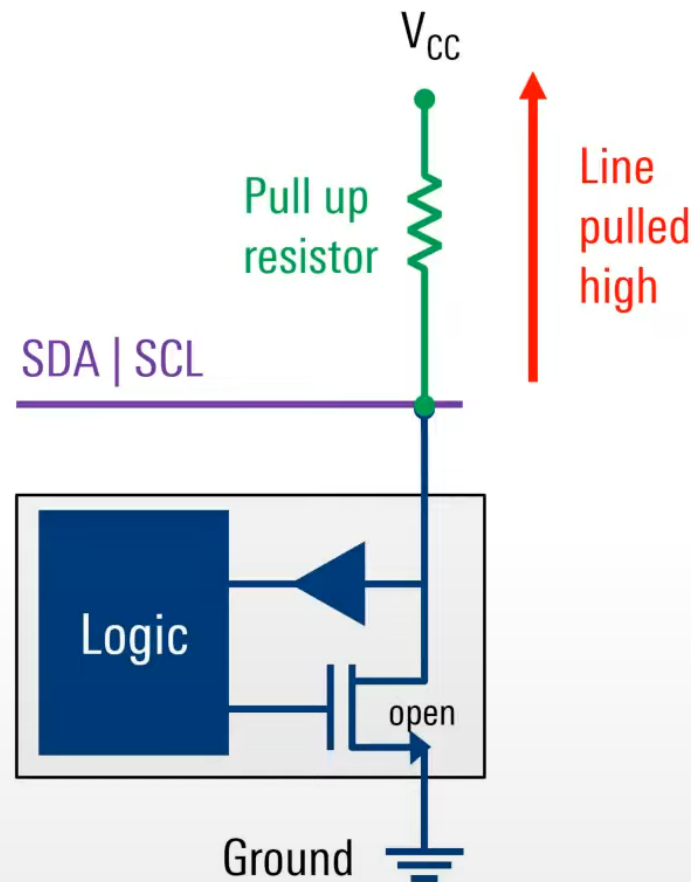
Stop condition

- ▶ Stop condition indicates the end of data bytes
 - First, SCL returns (and remains) high
 - Then, SDA returns (and remains) high
- ▶ Recall that for **data** bytes, SDA only transitions when clock is **low**
 - SDA transition when SCL **high** = **stop condition**
- ▶ Bus becomes idle
 - No clock signal
 - Any node can now use the start condition to claim the bus and begin a new communication



About “open drain”

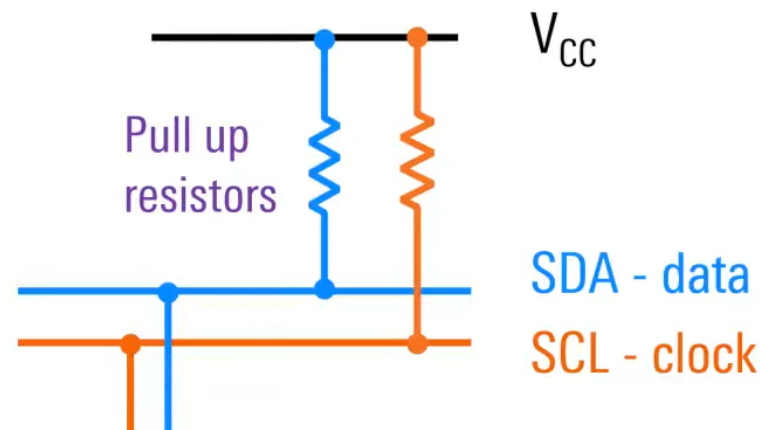
- ▶ Each line (SDA and SDL) is connected to voltage (V_{CC} or V_{DD}) via a “pull up” resistor
 - One resistor per line (not per device)
- ▶ Each I2C device contains logic that can open and close a drain
- ▶ When drain is “closed,” the line is pulled low (connected to ground)
- ▶ When drain is “open,” the line is pulled high (connected to voltage)
- ▶ I2C lines are high in the idle state
 - Sometimes called an “open drain” system



and therefore you may sometimes hear I2C referred to as an “open drain” system.

Pull up resistor values

- ▶ Pulling down a line is usually much faster than pulling up a line
 - Pull-up time is a function of bus capacitance and values of pull-up resistors
- ▶ Values of pull up resistors are a compromise
 - **Higher resistances** increase the time needed to pull up the line and thus **limit bus speed**
 - **Lower resistances** allow faster communications, but **require higher power**
- ▶ Typical pull-up resistor values are in the range of 1 k Ω – 10 k Ω



Modes / speeds

- ▶ I2C can operate at different bus speeds
 - Referred to as “modes”
- ▶ Table shows max speeds for each mode
 - Hardware is specified as compliant to standard, fast, or fast plus if it can (theoretically) achieve these speeds
- ▶ **High speed mode** devices are backwards compatible to lower speeds
 - Transmit a special sequence to switch the bus to HS mode
- ▶ **Ultra fast mode** is unidirectional (write only)
 - Also requires modification to the protocol and frames

I2C Mode	Speed
Standard Mode	100 kbps
Fast Mode	400 kbps
Fast Mode Plus	1 Mbps
High Speed Mode	3.4 Mbps
Ultra Fast Mode	5 Mbps

is, it is write-only, and also makes modifications

Understanding I2C to the standard I2C protocol and frames.

Summary

- ▶ I2C is a widely used protocol for exchanging data over short distances (intra-board)
- ▶ Two wires: serial data (SDA) and serial clock (SCL)
- ▶ A master seizes control of the bus and then reads or writes a frame containing a slave address and one or more bytes of data
 - Address and data bytes are individually acknowledged
 - Communication is initiated and terminated using special start and stop conditions
- ▶ I2C idle state is high (open drain)
- ▶ Several (maximum) bus speeds are supported

